

Avatar Cache System 設計書

Issue #416: アプリ再起動時にアバタースロットがリセットされる問題の解決と、再ロード高速化のための統合キャッシュシステム設計。

目次

1. 概要
2. 現状の問題
3. 設計目標
4. 設計思想
5. ディレクトリ構造
6. データ構造
7. キャッシュ対象
8. 処理フロー
9. エクスポート/インポート
10. 実装フェーズ

概要

アバターの読み込み高速化と永続化を実現するための統合キャッシュシステム。VRM/FBX ファイルからのパース・構築処理をスキップし、キャッシュから直接復元することで大幅な高速化を実現する。

期待される効果

項目	現状	キャッシュ後	改善率
初回ロード	3-8 秒	3-8 秒	-
再ロード	3-8 秒	0.5-1 秒	80-90%
テクスチャメモリ	40-80MB	4.5-9MB	89%

現状の問題

1. 永続化の問題

- OnApplicationPause(true) で SaveToFile() が呼ばれていない場合がある
- スワイプ終了時にデータが保存されない

2. 再ロードの非効率性

- 毎回 VRM/FBX をフルパース
- メッシュ・ボーンを毎回再構築
- テクスチャを毎回再圧縮（低スペック端末）

3. Asset 管理の分散

- アイコン、テクスチャ、ポーズ等が別々に管理
- 一貫性のあるマニフェストがない

設計目標

1. 高速再ロード: キャッシュから即時復元
2. 確実な永続化: アプリ終了時にデータを保存

3. 統合 Asset 管理: マニフェストで一元管理
4. 完全自己完結: キャッシュのみで再ロード可能 (元ファイル不要)
5. エクスポート対応: .avatacache / .unitypackage 形式で保存・共有
6. 拡張性: 将来の Asset 追加に対応

設計思想

スロットとキャッシュの分離

重要: スロットはキャッシュへの「参照」のみを保持する。

AvatarSlots/slots.json 参照 > AvatarCache/{hash}/manifest.json

メリット: - 同じアバターを複数スロットで共有可能 - キャッシュの重複を防止 - スロット削除時も他で使用中ならキャッシュは保持

キャッシュの完全自己完結性

キャッシュは元の VRM/FBX ファイルなしで再ロード可能。

データ	キャッシュ化	元ファイル依存
メッシュ (頂点、UV、ウェイト)	meshes.bin	不要
ボーン階層	bones.json	不要
Humanoid Avatar	humanoid.json	不要
テクスチャ	textures/*.astc	不要
マテリアル設定	materials.json	不要
BlendShape	blendshapes.bin	不要
VRM メタデータ	metadata.json	必須保存 (ライセンス)

ロード優先順位: 1. キャッシュから高速ロード (通常パス) 2. 元ファイルから再構築 (キャッシュ破損・バージョン不一致時) 3. 両方ない場合はエラー表示

端末別キャッシュ方針

全端末共通 (必須キャッシュ): 初回ロード時に必ず生成

キャッシュ	用途
manifest.json	キャッシュ管理
core/meshes.bin	メッシュ復元
core/bones.json	ボーン復元
core/humanoid.json	Avatar 復元
core/blendshapes.bin	BlendShape 復元
core/materials.json	マテリアル復元
icons/face.png	UI 表示
metadata.json	ライセンス情報

低スペック端末のみ (追加キャッシュ):

キャッシュ	用途
textures/*.astc	圧縮テクスチャ (メモリ削減)

ハイスぺック端末: - 必須キャッシュのみ生成 - テクスチャはメモリに余裕があるため非圧縮で保持 - 必要に応じて後から圧縮テクスチャを追加可能

ディレクトリ構造

{Application.persistentDataPath}/

```
AvatarSlots/
  slots.json          # スロット情報（パス参照のみ）

AvatarCache/
  {sourceFileHash}/   # キャッシュ本体（アバター単位）
                      # VRM/FBXファイルのハッシュで識別

  manifest.json       # アバターマニフェスト（自己完結情報）

  core/
    humanoid.json     # コアデータ（再構築必須）
    bones.json        # HumanDescription
    meshes.bin        # ボーン階層
    blendshapes.bin    # メッシュバイナリ
    materials.json    # BlendShapeバイナリ
                      # マテリアル設定

  textures/           # 圧縮テクスチャ
    tex_0.astc
    tex_1.astc
    ...

  icons/              # アイコン画像
    face.png          # 顔アイコン（512x512）

  poses/              # ポーズデータ（キャッシュ統合）
    manifest.json     # ポーズ一覧
    pose_0.png        # アイコン（256x256）
    pose_0.anim.bin   # AnimationClipバイナリ
    pose_1.png
    pose_1.anim.bin
    ...

  expressions/        # 表情データ（キャッシュ統合）
    manifest.json     # 表情一覧
    expr_0.png        # アイコン（256x256）
    expr_0.json       # BlendShape値
    expr_1.png
    expr_1.json
    ...

  metadata.json       # VRMメタ情報（ライセンス等）

Exports/
  MyAvatar.avatarcache # エクスポートファイル
  MyAvatar.unitypackage # 圧縮アーカイブ
                       # Unity パッケージ
```

データ構造

1. SlotsData (slots.json)

スロット管理。キャッシュへの参照のみ。

```
{
  "version": 1,
```

```

"slots": [
  {
    "slotIndex": 0,
    "cacheId": "a1b2c3d4e5f6...",
    "manifestPath": "AvatarCache/a1b2c3d4e5f6.../manifest.json",
    "lastUsedAt": "2026-01-24T15:30:00Z",
    "lastTransform": {
      "position": { "x": 0, "y": 0, "z": 0 },
      "rotation": { "x": 0, "y": 0, "z": 0, "w": 1 },
      "scale": { "x": 1, "y": 1, "z": 1 }
    }
  },
  {
    "slotIndex": 1,
    "cacheId": "a1b2c3d4e5f6...",
    "manifestPath": "AvatarCache/a1b2c3d4e5f6.../manifest.json",
    "lastUsedAt": "2026-01-24T14:00:00Z"
  }
],
"activeSlotIndex": 0
}

```

ポイント: - スロット 0 と 1 が同じキャッシュを参照可能 - lastTransform はスロットごとに独立

2. AvatarCacheManifest (manifest.json)

キャッシュの自己完結型マニフェスト。

```

{
  "manifestVersion": 1,
  "cacheFormatVersion": 1,

  "cacheId": "a1b2c3d4e5f6...",
  "avatarName": "MyAvatar",
  "createdAt": "2026-01-24T12:00:00Z",
  "lastAccessedAt": "2026-01-24T15:30:00Z",

  "sourceFile": {
    "originalPath": "/path/to/avatar.vrm",
    "hash": "sha256:abc123...",
    "fileSize": 52428800,
    "fileType": "VRM",
    "vrmVersion": "0.x",
    "required": false
  },

  "compatibility": {
    "unityVersion": "2022.3.x",
    "urpVersion": "14.x",
    "platform": "iOS"
  },

  "coreData": {
    "humanoidPath": "core/humanoid.json",
    "bonesPath": "core/bones.json",
    "meshesPath": "core/meshes.bin",
    "blendshapesPath": "core/blendshapes.bin",
    "materialsPath": "core/materials.json"
  }
}

```

```

},

"textures": [
  {
    "id": "tex_0",
    "path": "textures/tex_0.astc",
    "originalName": "body_diffuse",
    "format": "ASTC_6x6",
    "width": 2048,
    "height": 2048,
    "hash": "sha256:xxx"
  }
],

"icons": {
  "face": {
    "path": "icons/face.png",
    "width": 512,
    "height": 512
  }
},

"poses": {
  "manifestPath": "poses/manifest.json",
  "count": 2
},

"expressions": {
  "manifestPath": "expressions/manifest.json",
  "count": 5
},

"export": {
  "exportable": true,
  "lastExportedAt": null
}
}

```

3. VRMMetadata (metadata.json)

必須保存: VRM ライセンス情報（法的要件）

```

{
  "version": 1,
  "title": "MyAvatar",
  "author": "Author Name",
  "contactInformation": "contact@example.com",
  "reference": "https://example.com",

  "license": {
    "allowedUser": "OnlyAuthor",
    "violentUsage": "Disallow",
    "sexualUsage": "Disallow",
    "commercialUsage": "Disallow",
    "otherPermissionUrl": "",
    "licenseName": "CC_BY_NC_ND",
    "otherLicenseUrl": ""
  }
}

```

```

    },

    "thumbnail": "icons/face.png"
}

```

4. HumanoidCache (humanoid.json)

Unity Avatar 再構築用の HumanDescription 情報。

```

{
  "version": 1,
  "isHumanoid": true,
  "humanBones": [
    {
      "humanName": "Hips",
      "boneName": "J_Bip_C_Hips",
      "limit": {
        "useDefaultValues": true
      }
    },
    {
      "humanName": "Spine",
      "boneName": "J_Bip_C_Spine"
    }
  ],
  "skeleton": [
    {
      "name": "J_Bip_C_Hips",
      "position": { "x": 0, "y": 0.9, "z": 0 },
      "rotation": { "x": 0, "y": 0, "z": 0, "w": 1 },
      "scale": { "x": 1, "y": 1, "z": 1 }
    }
  ],
  "armStretch": 0.05,
  "legStretch": 0.05,
  "upperArmTwist": 0.5,
  "lowerArmTwist": 0.5,
  "upperLegTwist": 0.5,
  "lowerLegTwist": 0.5,
  "feetSpacing": 0,
  "hasTranslationDoF": false
}

```

5. BoneHierarchy (bones.json)

ボーン階層と Transform 情報。

```

{
  "version": 1,
  "rootBoneName": "Root",
  "bones": [
    {
      "index": 0,
      "name": "Root",
      "parentIndex": -1,
      "localPosition": { "x": 0, "y": 0, "z": 0 },
      "localRotation": { "x": 0, "y": 0, "z": 0, "w": 1 },
      "localScale": { "x": 1, "y": 1, "z": 1 }
    }
  ]
}

```

```

    },
    {
        "index": 1,
        "name": "J_Bip_C_Hips",
        "parentIndex": 0,
        "localPosition": { "x": 0, "y": 0.9, "z": 0 },
        "localRotation": { "x": 0, "y": 0, "z": 0, "w": 1 },
        "localScale": { "x": 1, "y": 1, "z": 1 }
    }
]
}

```

6. MeshCache (meshes.bin)

バイナリ形式のメッシュデータ。

```

[Header]
- Magic: "MESH" (4 bytes)
- Version: uint32
- MeshCount: uint32

[For each Mesh]
- NameLength: uint32
- Name: string (UTF-8)
- VertexCount: uint32
- Vertices: float32[] (x,y,z × VertexCount)
- HasNormals: bool
- Normals: float32[] (if HasNormals)
- HasTangents: bool
- Tangents: float32[] (x,y,z,w × VertexCount, if HasTangents)
- UVChannelCount: uint32
- UVs[]: float32[] (u,v × VertexCount × UVChannelCount)
- HasColors: bool
- Colors: float32[] (r,g,b,a × VertexCount, if HasColors)
- SubMeshCount: uint32
- SubMeshes: { topology, indexStart, indexCount, baseVertex }[]
- Triangles: uint32[]
- HasBoneWeights: bool
- BoneWeights: { boneIndex0-3, weight0-3 }[] (if HasBoneWeights)
- BindPoseCount: uint32
- BindPoses: Matrix4x4[] (if HasBoneWeights)

```

7. BlendShapeCache (blendshapes.bin)

```

[Header]
- Magic: "BLND" (4 bytes)
- Version: uint32
- MeshCount: uint32

```

```

[For each Mesh]
- MeshNameLength: uint32
- MeshName: string
- BlendShapeCount: uint32

```

```

[For each BlendShape]
- NameLength: uint32
- Name: string
- FrameCount: uint32

```

```
[For each Frame]
- Weight: float32
- DeltaVertices: float32[] (x,y,z × VertexCount)
- DeltaNormals: float32[] (x,y,z × VertexCount)
- DeltaTangents: float32[] (x,y,z × VertexCount)
```

8. MaterialCache (materials.json)

マテリアル情報（テクスチャ参照含む）。

```
{
  "version": 1,
  "materials": [
    {
      "name": "Body",
      "shaderName": "VRM/MToon",
      "renderQueue": 2000,
      "keywords": ["_NORMALMAP", "_EMISSION"],
      "properties": {
        "_Color": { "type": "Color", "value": { "r": 1, "g": 1, "b": 1, "a": 1 } },
        "_MainTex": { "type": "Texture", "value": "tex_0" },
        "_BumpMap": { "type": "Texture", "value": "tex_1" },
        "_ShadeColor": { "type": "Color", "value": { "r": 0.8, "g": 0.8, "b": 0.8, "a": 1 } },
        "_Cutoff": { "type": "Float", "value": 0.5 }
      }
    }
  ],
  "meshMaterialMapping": [
    {
      "meshName": "Body",
      "submeshMaterials": ["Body", "Face"]
    }
  ]
}
```

9. PoseManifest (poses/manifest.json)

ポーズデータ管理。AnimationClip をバイナリ形式で保存。

```
{
  "version": 1,
  "poses": [
    {
      "index": 0,
      "name": "Default Pose",
      "iconPath": "pose_0.png",
      "animationPath": "pose_0.anim.bin",
      "isDefault": true,
      "createdAt": "2026-01-24T12:00:00Z"
    },
    {
      "index": 1,
      "name": "Victory Pose",
      "iconPath": "pose_1.png",
      "animationPath": "pose_1.anim.bin",
      "isDefault": false,
      "createdAt": "2026-01-24T13:00:00Z"
    }
  ]
}
```



```

    }
  ]
}

```

10. ExpressionManifest (expressions/manifest.json)

表情データ管理。BlendShape 値を JSON 形式で保存。

```

{
  "version": 1,
  "expressions": [
    {
      "index": 0,
      "name": "Happy",
      "preset": "Joy",
      "iconPath": "expr_0.png",
      "dataPath": "expr_0.json",
      "createdAt": "2026-01-24T12:00:00Z"
    },
    {
      "index": 1,
      "name": "Sad",
      "preset": "Sorrow",
      "iconPath": "expr_1.png",
      "dataPath": "expr_1.json",
      "createdAt": "2026-01-24T12:00:00Z"
    }
  ]
}

```

11. ExpressionData (expressions/expr_*.json)

個別の表情データ。

```

{
  "version": 1,
  "name": "Happy",
  "preset": "Joy",
  "blendShapeValues": {
    "Face.M_F00_000_Fcl_ALL_Joy": 1.0,
    "Face.M_F00_000_Fcl_EYE_Close": 0.3
  },
  "materialColorOverrides": {
    "Face": {
      "_Color": { "r": 1.0, "g": 0.95, "b": 0.95, "a": 1.0 }
    }
  }
}

```

12. AnimationCache (poses/pose_*.anim.bin)

AnimationClip のバイナリ形式。

```

[Header]
- Magic: "ANIM" (4 bytes)
- Version: uint32
- ClipName: string
- FrameRate: float32
- Length: float32

```

```

- WrapMode: uint32

[Curves]
- CurveCount: uint32

[For each Curve]
- PathLength: uint32
- Path: string (e.g., "J_Bip_C_Hips")
- PropertyName: string (e.g., "localPosition.x")
- KeyframeCount: uint32

[For each Keyframe]
- Time: float32
- Value: float32
- InTangent: float32
- OutTangent: float32
- WeightedMode: uint32

```

キャッシュ対象

優先度: 高 (Phase 1-3)

対象	形式	サイズ目安	効果
Mesh データ	Binary	1-10MB	パース時間削減
BlendShape	Binary	0.5-5MB	表情再構築不要
ボーン階層	JSON	10-50KB	構築時間削減
HumanDescription	JSON	5-20KB	Avatar 再構築高速化
圧縮テクスチャ	ASTC/ETC2	1-10MB	再圧縮不要

優先度: 中 (Phase 4-5)

対象	形式	サイズ目安	効果
アバターアイコン	PNG	50-200KB	即時表示
マテリアル情報	JSON	5-20KB	設定復元
VRM メタ情報	JSON	1-5KB	ライセンス表示

優先度: 低 (Phase 7-8)

対象	形式	サイズ目安	効果
ポーズアイコン	PNG	20-50KB	UI 表示
ポーズデータ	Binary	10-100KB	AnimationClip 復元
表情アイコン	PNG	20-50KB	UI 表示
表情データ	JSON	1-5KB	BlendShape 復元

処理フロー

初回ロード

VRM/FBX ファイル選択

↓

```

ファイルハッシュ計算 (SHA256)
↓
キャッシュ存在チェック → なし
↓
通常ロード (UniVRM/Assimp)
↓
GameObject 構築
↓
【必須キャッシュ生成 (全端末共通・非同期)】
manifest.json 生成
core/humanoid.json 保存
core/bones.json 保存
core/meshes.bin 保存
core/blendshapes.bin 保存
core/materials.json 保存
metadata.json 保存 (VRMのみ)
icons/face.png 撮影・保存
↓
【追加キャッシュ生成 (低スペック端末のみ)】
textures/*.astc 圧縮・保存
↓
スロットに登録 (manifestPathを参照)
↓
slots.json 保存

再ロード (キャッシュあり)
スロット選択
↓
slots.json からマニフェストパス取得
↓
manifest.json 読み込み
↓
cacheFormatVersion 確認
↓ (互換性OK)
【高速ロードパス】
bones.json → ボーン階層構築
humanoid.json → Avatar 構築
meshes.bin → Mesh 構築
blendshapes.bin → BlendShape 適用
materials.json + textures/ → マテリアル適用
icons/face.png → アイコン表示
↓
GameObject 完成 (0.5-1秒)
↓
slots.json から位置情報復元

キャッシュ不整合時
manifest.json 読み込み
↓
cacheFormatVersion 確認
↓ (不一致 or 破損)
sourceFile.originalPath から元ファイル存在確認
↓
存在する → 初回ロードフローへ (キャッシュ再生成)
存在しない → エラー表示「元ファイルが見つかりません」

```

エクスポート/インポート

エクスポート形式

形式	拡張子	用途
Avatar Cache Archive	<code>.avatacache</code>	バックアップ、端末間移行、配布
Unity Package	<code>.unitypackage</code>	Unity プロジェクト間共有

`.avatacache` 形式

ZIP 形式のアーカイブ（独自拡張子）

MyAvatar.avatacache (ZIP)

```
manifest.json
core/
  humanoid.json
  bones.json
  meshes.bin
  blendshapes.bin
  materials.json
textures/
  *.astc
icons/
  face.png
poses/
  manifest.json
  *.png          # アイコン
  *.anim.bin     # AnimationClip データ
expressions/
  manifest.json
  *.png          # アイコン
  *.json         # BlendShape データ
metadata.json
```

インポート処理

`.avatacache` ファイル 選択

↓

ZIP 展開（一時領域）

↓

`manifest.json` 読み込み

↓

互換性チェック（`cacheFormatVersion`, `platform`）

↓

互換性OK → `AvatarCache/` にコピー

プラットフォーム不一致 → テクスチャ再圧縮

バージョン不一致 → エラー or マイグレーション

↓

`slots.json` に登録

↓

ロード

将来の拡張

- クラウド同期: iCloud / Google Drive 連携

- プリセット配布: 公式アバターの配布
 - アバターショップ: .avatarcache 形式での販売
-

実装フェーズ

Phase 1: マニフェスト・スロット分離

目的: スロットとキャッシュの分離、基盤構築

タスク: - [] SlotsData クラス作成 (参照のみ) - [] AvatarCacheManifest クラス作成 - [] ファイルハッシュ計算ユーティリティ - [] キャッシュディレクトリ管理

成果物: - SlotsData.cs - AvatarCacheManifest.cs - AvatarCacheManager.cs

Phase 2: ボーン/Humanoid キャッシュ

目的: ボーン構築の高速化

タスク: - [] HumanoidCacheSerializer クラス作成 - [] BoneHierarchyCacheSerializer クラス作成 - [] HumanDescription のシリアライズ/デシリアライズ - [] キャッシュからのボーン再構築

成果物: - HumanoidCacheSerializer.cs - BoneHierarchyCacheSerializer.cs

Phase 3: Mesh/BlendShape キャッシュ

目的: メッシュ構築の高速化

タスク: - [] MeshCacheSerializer クラス作成 - [] BlendShapeCacheSerializer クラス作成 - [] バイナリ形式の実装 - [] キャッシュからのメッシュ再構築

成果物: - MeshCacheSerializer.cs - BlendShapeCacheSerializer.cs

Phase 4: テクスチャ/マテリアル キャッシュ

目的: 圧縮テクスチャの再利用

タスク: - [] TextureCacheManager クラス作成 - [] MaterialCacheSerializer クラス作成 - [] 圧縮テクスチャのファイル保存/読み込み - [] マテリアル再構築

成果物: - TextureCacheManager.cs - MaterialCacheSerializer.cs

Phase 5: 高速ロードバス実装

目的: キャッシュからの即時復元

タスク: - [] AvatarCacheLoader クラス作成 - [] キャッシュ存在チェック - [] 高速ロードバスの実装 - [] 既存ローダーとの統合 - [] フォールバック処理

成果物: - AvatarCacheLoader.cs - RuntimeFBXLoaderBridge.cs 修正

Phase 6: 永続化・保存タイミング修正

目的: 確実な永続化

タスク: - [] OnApplicationPause での確実な保存 - [] OnApplicationQuit でのバックアップ - [] キャッシュ生成の非同期化 - [] エラーハンドリング強化

成果物: - AvatarSlotManager.cs 修正 - PersistenceManager.cs

Phase 7: エクスポート/インポート

目的: キャッシュの移植性

タスク: - [] .avatacache エクスポート実装 - [] .avatacache インポート実装 - [] 互換性チェック - [] マイグレーション処理

成果物: - AvatarCacheExporter.cs - AvatarCacheImporter.cs

Phase 8: ポーズ/表情アイコン (将来対応)

目的: UI 向けアイコン管理

タスク: - [] ポーズアイコン生成・保存 - [] 表情アイコン生成・保存 - [] マニフェストとの連携

関連ファイル

既存ファイル (修正対象)

- Assets/Scripts/FBXLoader/Core/AvatarManifest.cs
- Assets/Scripts/FBXLoader/Core/AvatarSlotData.cs
- Assets/Scripts/FBXLoader/Core/AvatarSlotManager.cs
- Assets/Scripts/FBXLoader/Core/AvatarMemoryCache.cs
- Assets/Scripts/FBXLoader/Core/RuntimeFBXLoaderBridge.cs

新規ファイル (作成予定)

```
Assets/Scripts/FBXLoader/Cache/  
  Core/  
    SlotsData.cs  
    AvatarCacheManifest.cs  
    AvatarCacheManager.cs  
  Serializers/  
    HumanoidCacheSerializer.cs  
    BoneHierarchyCacheSerializer.cs  
    MeshCacheSerializer.cs  
    BlendShapeCacheSerializer.cs  
    MaterialCacheSerializer.cs  
    TextureCacheManager.cs  
  Loader/  
    AvatarCacheLoader.cs  
  Export/  
    AvatarCacheExporter.cs  
    AvatarCacheImporter.cs  
  Persistence/  
    PersistenceManager.cs
```

参考

- Issue #416: アプリ再起動時にアバタースロットがリセットされる
- Issue #440: 低スペック端末への最適化
- **AvatarLoaderOptimization.md**: 既存の最適化ドキュメント